

Module 15 - chapter 12

Ansible Variables - make your Playbook customizable

The project files used in this lecture can be found here: <https://gitlab.com/twn-devops-bootcamp/latest/15-ansible/deploy-node-app/-/tree/feature/variables>

Registered variables

In the previous chapter (Module 15 - chapter 11) we used registered variables

```
34     npm:
35         path: /home/nana/package
36     - name: Start the application
37       command:
38         chdir: /home/nana/package/app
39         cmd: node server
40     async: 1000
41     poll: 0
42     - name: Ensure app is running
43       shell: ps aux | grep node
44       register: app_status
45     - debug: msg={{app_status.stdout_lines}}
46
```

Registering variables

- ▶ Create variables from the output of an Ansible task
- ▶ This variable can be used in any later tasks in your play



We can get the results of other tasks for example the unarchive and use a registered variable to store the result of the task and then print it in the terminal using the 'debug' module

```
27 become: true
28 become_user: nana
29 tasks:
30   - name: Unpack the nodejs file
31     unarchive:
32       src: /Users/nana/ansible/nodejs-app/nodejs-app-1.0.0.tgz
33       dest: /home/nana
34       register: user_creation_result
35   - debug: msg={{user_creation_result}}
36   - name: Install dependencies
37     npm:
38       path: /home/nana/package
39   - name: Start the application

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

ok: [165.22.22.94]

TASK [debug] *****
ok: [165.22.22.94] => {
  "msg": {
    "changed": false,
    "dest": "/home/nana",
    "failed": false,
    "gid": 114,
    "group": "admin",
    "handler": "TgzArchive",
    "mode": "0750",
    "owner": "nana",
    "size": 4096,
    "src": "/var/tmp/ansible-tmp-1703006243.192766-14393-100001150503750/source",
    "state": "directory",
    "uid": 1000
  }
}

TASK [Install dependencies] *****
ok: [165.22.22.94]
```

Not all of the modules will have documented the return information. If they do then we can see it in the documentation for example the 'command' module

docs.ansible.com/projects/ansible-core/devel/collections/ansible/builtin/command_module.html#ansible-collections-ansible-builtin-command-module

Channel 4 BBC iPlayer - Home Trello Wise - Login Google NotebookLM Overleaf - CV CodeSignal Learn Cloud Linode Linux Bash coding Java DevOps

Ansible Core Documentation

Galaxy Developer Guide

REFERENCE & APPENDICES

Collection Index

- Collections in the Ansible Namespace
 - Ansible.Builtin
 - Description
 - Communication
 - Plugin Index
- Indexes of all modules and plugins
- Playbook Keywords
- Return Values
- Ansible Configuration Settings
- Controlling how Ansible behaves: precedence rules
- YAML Syntax
- Python 3 Support
- Interpreter Discovery
- Releases and maintenance
- Testing Strategies
- Sanity Tests
- Frequently Asked Questions
- Glossary
- Ansible Reference: Module Utilities
- Special Variables
- Red Hat Ansible Automation Platform
- Ansible Automation Hub
- Logging Ansible output

```
- --debug
- --longopt
- value for longopt
- --other-longopt=value for other longopt
- positional

- name: Safely use templated variable to run command. Always use the quote filter to avoid injection issues
  ansible.builtin.command: cat {{ myfile|quote }}
  register: myoutput
```

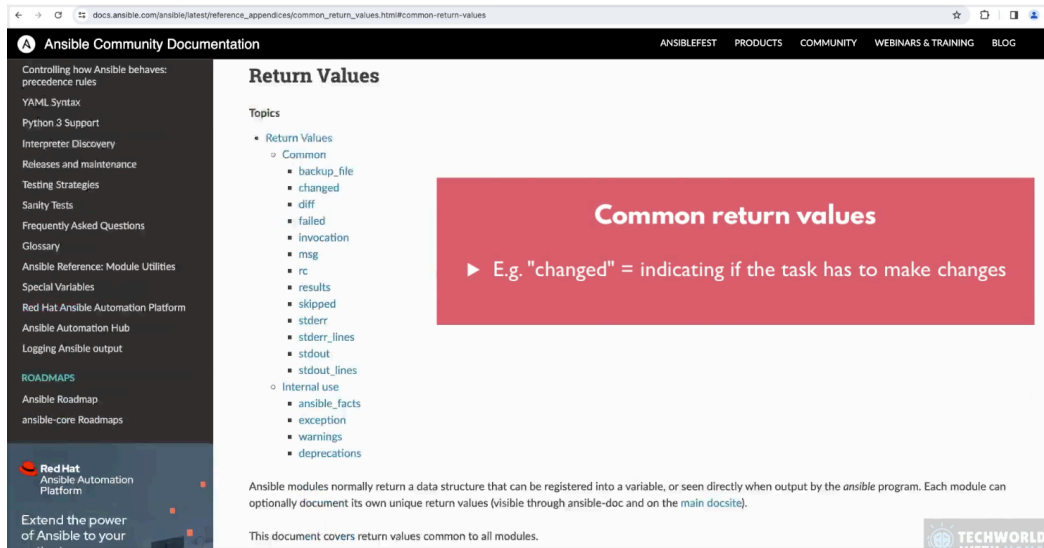
Return Values

Common return values are documented [here](#), the following are the fields unique to this module:

Key	Description
cmd list / elements=string	The command executed by the task. Returned: always Sample: ["echo", "hello"]
delta string	The command execution delta time. Returned: always Sample: "0:00:00.001529"
end string	The command execution end time. Returned: always Sample: "2017-09-29 22:03:40.004657"
msg boolean	changed Returned: always Sample: true
rc integer	The command return code (0 means success). Returned: always Sample: 0
start % string	The command execution start time.

And if we click on 'here' (in blue font in the screenshot above) then we go to

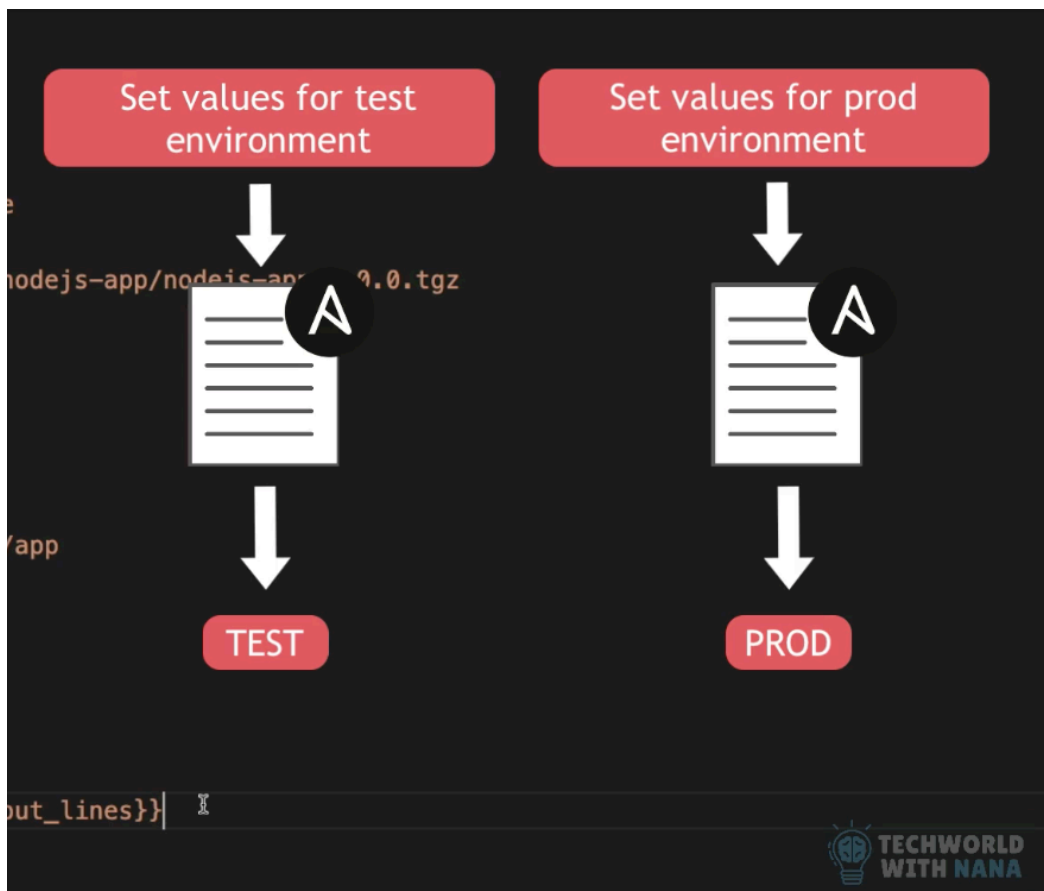
the list of Common return values:



Nana suggest is better to print the whole result in the terminal and see what we get instead of checking them the documentation.

Parameterise Playbook

We want to make the values configurable per environment for example



We will use double curly braces

```
23 - name: Deploy nodejs app
24   hosts: 165.22.22.94
25   become: True
26   become_user: nana
27   tasks:
28     - name: Unpack the nodejs file
29       unarchive:
30         src: '{{node-file-location}}'
31         dest: /home/nana
32     - name: Install dependencies
33       npm:
34         path: /home/nana/package
35     - name: Start the application
36       command:
37         chdir: /home/nana/package/app
38         cmd: node server
39       async: 1000
40       poll: 0
41     - name: Ensure app is running
42       shell: ps aux | grep node
43       register: app_status
44     - debug: msg={{app_status.stdout_lines}}
45
```

Referencing variables

- ▶ Using double curly braces
- ▶ If you start a value with {{ value }}, you must quote the whole expression to create valid YAML syntax



but if the curly braces go immediately after a colon we will need quotation marks, because ansible will think that it refers to a yaml dictionary syntax instead of a variable syntax.

```
27   tasks:
28     - name: Unpack the nodejs file
29       unarchive:
30         src: '{{node-file-location}}'
31         dest: /home/nana
```

If the curly braces are not immediately after a colon then we don't need the quotation marks:

```
27   tasks:
28     - name: Unpack the nodejs file
29       unarchive:
30         src: /Users/nana/ansible/nodejs-app/nodejs-app-{{version}}.tgz
31         dest: /home/nana
```

If we need to parameterise part of the path

```
29       unarchive:
30         src: '{{location}}/nodejs-app-{{version}}.tgz
```

then we use "" but for the whole line

```
29     unpack:
30       src: "{{location}}/nodejs-app-{{version}}.tgz"
```

This won't work:

```
29     unpack:
30       src: "{{location}}"/nodejs-app-{{version}}.tgz
```

We can also parameterise '/home/astrid/' as it is repeated

```
26   tasks:
27     - name: Unpack the nodejs file
28       unpack:
29         src: "{{location}}/nodejs-app/nodejs-app-{{version}}.tgz"
30         dest: /home/astrid
31     - name: Install dependencies
32       npm:
33         path: /home/astrid/package
34     - name: Start the application
35       command:
36         chdir: /home/astrid/package/app
37         cmd: node server
38       async: 1000
39       poll: 0
40     - name: Ensure app is running
41       shell: ps aux | grep node
42       register: app_status
43     - debug: msg={{app_status.stdout_lines}}
```

And now we have:

```

26 tasks:
27   - name: Unpack the nodejs file
28     unarchive:
29       src: "{{location}}/nodejs-app/nodejs-app-{{version}}.tgz"
30       dest: "{{destination}}"
31   - name: Install dependencies
32     npm:
33       path: "{{destination}}/package"
34   - name: Start the application
35     command:
36       chdir: "{{destination}}/package/app"
37       cmd: node server
38       async: 1000
39       poll: 0
40   - name: Ensure app is running
41     shell: ps aux | grep node
42     register: app_status
43   - debug: msg={{app_status.stdout_lines}}
44

```

Now we need to set the values, if we don't and execute the ansible playbook we will get an error

-> ansible-playbook -i hosts deploy-node.yaml

```

TASK [Unpack the nodejs file] *****
[ERROR]: Task failed: Finalization of task args for 'ansible.builtin.unarchive' failed: Error while resolving value for 'dest':
'destination' is undefined

Task failed.
Origin: /Users/astrid/PythonProject/Ansible/deploy-node.yaml:27:7

25  become_user: astrid
26  tasks:
27    - name: Unpack the nodejs file
      ^ column 7

<<< caused by >>>

Finalization of task args for 'ansible.builtin.unarchive' failed.
Origin: /Users/astrid/PythonProject/Ansible/deploy-node.yaml:28:7

```

Variables defined in a playbook

One way is to set them up under 'vars' attribute in the play and add the variables inside a dictionary structure:

```

22  - name: Deploy nodejs app
23    hosts: 46.101.49.110
24    become: true
25    become_user: astrid
26    vars:
27
28    tasks:
29      - name: Unpack the nodejs file
30        unarchive:
31          src: "{{location}}/nodejs-app-{{version}}.tgz"
32          dest: "{{destination}}"
33      - name: Install dependencies
34        npm:
35          path: "{{destination}}/package"
36      - name: Start the application
37        command:
38          chdir: "{{destination}}/package/app"
39          cmd: node server

```

```

22  - name: Deploy nodejs app
23    hosts: 46.101.49.110
24    become: true
25    become_user: astrid
26    vars: {
27      location: /Users/astrid/PythonProject/Ansible/nodejs-app,
28      version: 1.0.0,
29      destination: /home/astrid
30    }
31    tasks:
32      - name: Unpack the nodejs file
33        unarchive:
34          src: "{{location}}/nodejs-app-{{version}}.tgz"
35          dest: "{{destination}}"
36      - name: Install dependencies
37        npm:
38          path: "{{destination}}/package"
39      - name: Start the application
40        command:
41          chdir: "{{destination}}/package/app"
42          cmd: node server

```

And if I execute it

-> ansible-playbook -i hosts deploy-node.yaml

```

TASK [Ensure app is running] *****
changed: [46.101.49.110]

TASK [debug] *****
ok: [46.101.49.110] => {
  "msg": [
    "astrid    41649 50.0  6.1 618048 60676 ?      Rl   14:29   0:00 node server",
    "astrid    41664  0.0  0.1  2800   1916 pts/1    S+   14:29   0:00 /bin/sh -c ps aux | grep node",
    "astrid    41666  0.0  0.2  7076   2164 pts/1    S+   14:29   0:00 grep node"
  ]
}

PLAY RECAP *****
46.101.49.110      : ok=11  changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

```

The second way is to make the playbook properly configurable instead of hardcoding the values inside the play.

Passing Variables On The command line

We will define the variables outside the playbook!

So let's say we want the location and the version to be set outside, so we removed them from 'vars'

```

22  - name: Deploy nodejs app
23      hosts: 46.101.49.110
24      become: true
25      become_user: astrid
26      vars: {}
27          destination: /home/astrid
28      }
29      tasks:
30          - name: Unpack the nodejs file
31              unarchive:

```

and then we pass the values at the moment we run the command to execute the ansible playbook along with a flag '--extra-vars' or its shortcut '-e'

-> ansible-playbook -i hosts deploy-node.yaml --extra-vars

-> ansible-playbook -i hosts deploy-node.yaml -e

Along with the variables as a key:value pair

-> ansible-playbook -i hosts deploy-node.yaml -e "version=1.0.0 location=/Users/astrid/PythonProject/Ansible/nodejs-app"

```
> ansible-playbook -i hosts deploy-node.yaml -e "version=1.0.0 location=/Users/astrid/PythonProject/Ansible/nodejs-app"

PLAY [Install node and npm] *****

TASK [Gathering Facts] *****
ok: [46.101.49.110]

TASK [Update apt repo and cache] *****
ok: [46.101.49.110]

TASK [Install nodejs and npm] *****
ok: [46.101.49.110]

PLAY [Create new linux user for node app] *****

TASK [Gathering Facts] *****
ok: [46.101.49.110]

TASK [Create linux user] *****
ok: [46.101.49.110]

PLAY [Deploy nodejs app] *****

TASK [Gathering Facts] *****
ok: [46.101.49.110]

TASK [Unpack the nodejs file] *****
ok: [46.101.49.110]

TASK [Install dependencies] *****
ok: [46.101.49.110]

TASK [Start the application] *****
changed: [46.101.49.110]

TASK [Ensure app is running] *****
changed: [46.101.49.110]

TASK [debug] *****
ok: [46.101.49.110] => {
  "msg": [
    "astrid    41649  0.1  6.0 618304 59280 ?        Sl  14:29   0:00 node server",
    "astrid    42249  0.0  0.1  2800   1912 pts/1   S+  14:39   0:00 /bin/sh -c ps aux | grep node",
    "astrid    42251  0.0  0.2   7076   2168 pts/1   S+  14:39   0:00 grep node"
  ]
}

PLAY RECAP *****
46.101.49.110      : ok=11  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

We want to use the user name 'astrid' and it is repeated in the current yaml. If we use the first way to set the value 'astrid' inside the playbook we will have to create add the 'name' in each play that we need it (See lines 17 and line 30)

```
15  hosts: 165.22.22.94
16  vars:
17  |   name: nana
18  tasks:
19  |   - name: Create linux user
20  |     user:
21  |       name: nana
22  |       comment: Nana Admin
23  |       group: admin
24
25  - name: Deploy nodejs app
26    hosts: 165.22.22.94
27    become: True
28    become_user: nana
29    vars:
30    |   name: nana
31    |   destination: /home/nana
32    tasks:
33    |   - name: Unpack the nodejs file
```

This is not efficient, it is better to pass it as argument in the terminal.

So let's refactor:

```

15 tasks:
16   - name: Create linux user
17     user:
18       name: "{{name}}"
19       comment: Astrid admin
20       group: admin
21
22   - name: Deploy nodejs app
23     hosts: 46.101.49.110
24     become: true
25     become_user: "{{name}}"
26     vars: {
27       destination: "/home/{{name}}"
28     }
29 tasks:

```

And now we have the variable 'name' but is not defined or hardcoded in each playbook.

Now we can execute the playbook and we need to pass the name as argument and actually let's use 'node' as user instead of 'astrid'

-> `ansible-playbook -i hosts deploy-node.yaml -e "version=1.0.0 location=/Users/astrid/PythonProject/Ansible/nodejs-app name=node"`

```

TASK [debug] *****
ok: [46.101.49.110] => {
  "msg": [
    "node      42935  0.0  2.1  40912 21072 ?        S   14:52   0:00 /usr/bin/python3.12 /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/async_wrapper.py j221932103497 1000 false /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/AnsiballZ_command.py /usr/bin/python3.12 /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/AnsiballZ_command.py",
    "node      42936  0.0  2.1  40912 21332 ?        S   14:52   0:00 /usr/bin/python3.12 /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/async_wrapper.py j221932103497 1000 false /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/AnsiballZ_command.py /usr/bin/python3.12 /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/AnsiballZ_command.py",
    "node      42937 22.2  2.9  40416 28584 ?        S   14:52   0:00 /usr/bin/python3.12 /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/AnsiballZ_command.py",
    "node      42938 51.5  6.4 618056 63516 ?        Sl  14:52   0:00 node server",
    "root      42962  0.0  0.1   2800   1848 pts/1    Ss+  14:52   0:00 /bin/sh -c sudo -H -S -n -u node /bin/sh -c 'echo BECOME-SUCCESS-agvljtlylxtplmxqzbohnbhvaummzr ; /usr/bin/python3.12 /var/tmp/ansible-tmp-1785941576.5659258-14514-42603695051684/AnsiballZ_command.py'",
    "root      42963  0.0  0.7  16948   7012 pts/1    S+   14:52   0:00 sudo -H -S -n -u node /bin/sh -c echo BECOME-SUCCESS-agvljtlylxtplmxqzbohnbhvaummzr ; /usr/bin/python3.12 /var/tmp/ansible-tmp-1785941576.5659258-14514-42603695051684/AnsiballZ_command.py",
    "root      42964  0.0  0.2  16948   2572 pts/2    Ss   14:52   0:00 sudo -H -S -n -u node /bin/sh -c echo BECOME-SUCCESS-agvljtlylxtplmxqzbohnbhvaummzr ; /usr/bin/python3.12 /var/tmp/ansible-tmp-1785941576.5659258-14514-42603695051684/Ansiball

```

The log is longer because we called the new user 'node' and due to that there

are several processes with 'node' so 'grep node' brings them too

Here is the same result but from the droplets console

```
root@nodejs-app1:~# ps aux | grep node
node    42935  0.0  2.1 40912 21072 ?        S    14:52   0:00 /usr/bin/python3.12 /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/async_wrapper.py j221932103497 1000 false /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/Ansiballf_command.py /usr/bin/python3.12 /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/Ansiballf_command.py
node    42936  0.0  2.1 40912 21332 ?        S    14:52   0:00 /usr/bin/python3.12 /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/async_wrapper.py j221932103497 1000 false /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/Ansiballf_command.py /usr/bin/python3.12 /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/Ansiballf_command.py
node    42937  0.0  2.9 40416 38584 ?        S    14:52   0:00 /usr/bin/python3.12 /var/tmp/ansible-tmp-1785941574.9672909-14502-50420489012334/Ansiballf_command.py
node    42938  1.6  5.9 618312 58392 ?        sl   14:52   0:00 node server
root    42977  0.0  0.2  7076  2168 pts/0    S+   14:53   0:00 grep --color=auto node
root@nodejs-app1:~#
```

We also got this warning

```
) ansible-playbook -i hosts deploy-node.yaml -e "version=1.0.0 location=/Users/astrid/PythonProject/Ansible/nodejs-app name=node"
[WARNING]: Found variable using reserved name 'name'.
Origin: <CLI option '-e'>

name

PLAY [Install node and npm] *****
```

Regarding the use of a 'reserved word' and we should avoid doing that

Naming of Variables

- ▶ Not valid: Python keywords, such as *async*
Playbook keywords, such as *environment*
- ▶ Valid: letters, numbers and underscores
- ▶ Should always start with a letter
- ▶ **DON'T**: linux-name, linux name, linux.name or l2

Let's refactor

```
13  - name: Create new linux user for node app
14    hosts: 46.101.49.110
15    tasks:
16      - name: Create linux user
17        user:
18          name: "{{linux_name}}"
19          comment: Astrid admin
20          group: admin
21
22  - name: Deploy nodejs app
23    hosts: 46.101.49.110
24    become: true
25    become_user: "{{linux_name}}"
26    vars: {
27      destination: "/home/{{linux_name}}"
28    }
29    tasks:
30      - name: Unpack the nodejs file
```

Same for 'destination' it is too vague.

```

22 - name: Deploy nodejs app
23   hosts: 46.101.49.110
24   become: true
25   become_user: "{{linux_name}}"
26   vars: {
27     user_home_dir: "/home/{{linux_name}}"
28   }
29   tasks:
30     - name: Unpack the nodejs file
31       unarchive:
32         src: "{{location}}/nodejs-app-{{version}}.tgz"
33         dest: "{{user_home_dir}}"
34     - name: Install dependencies
35       npm:
36         path: "{{user_home_dir}}/package"
37     - name: Start the application
38       command:
39         chdir: "{{user_home_dir}}/package/app"
40         cmd: node server
41       async: 1000

```

External Variables File

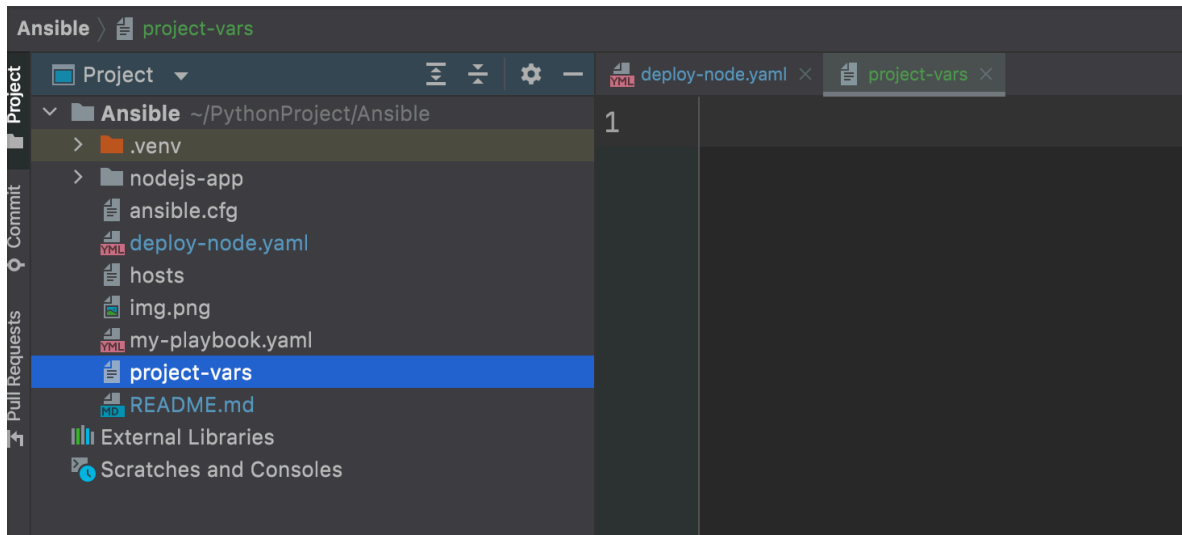
But if I have too many parameters it is not convenient to pass all those arguments in the terminal.

The solution is to use a separate variables file

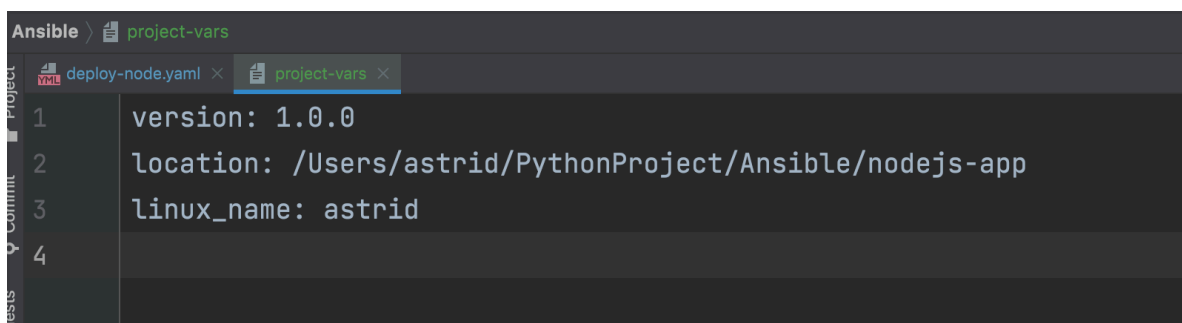
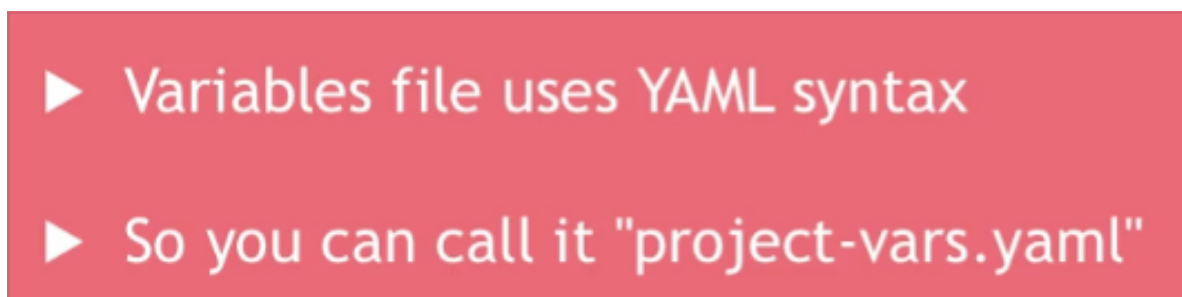


Then we will tell Ansible to use the vars file.

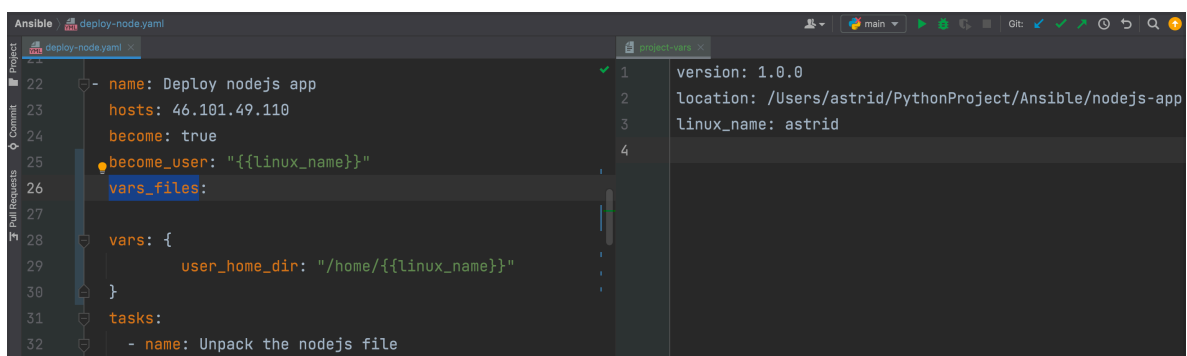
So I created a new text file (it can also be a yaml file) and I called it 'project-vars' or anything I want



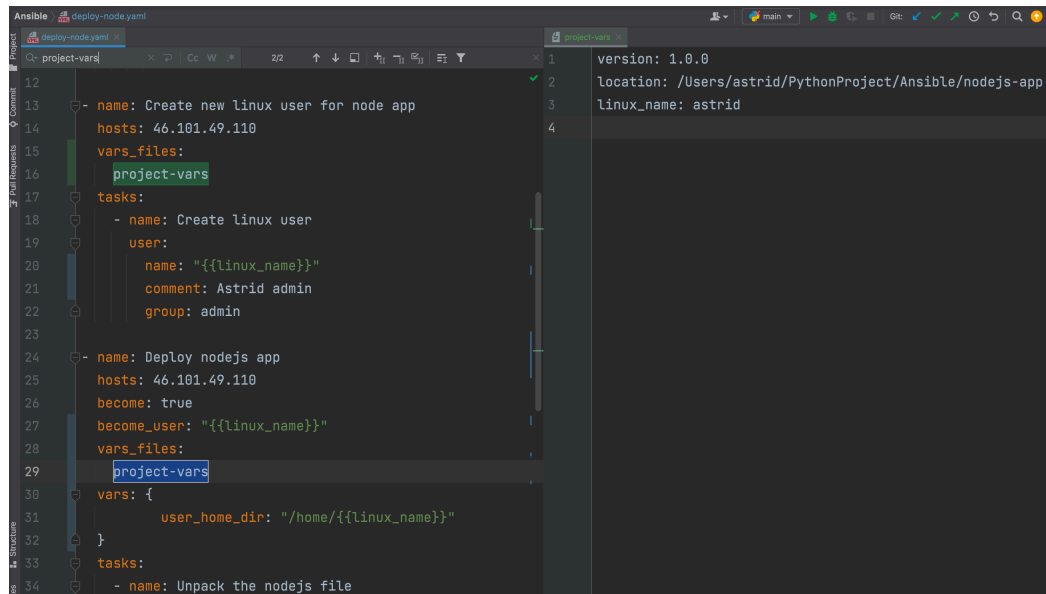
And then I write the variables and their values



And then I reference the project-vars file inside the playbook using the attribute 'vars_files'



in each play I need it:



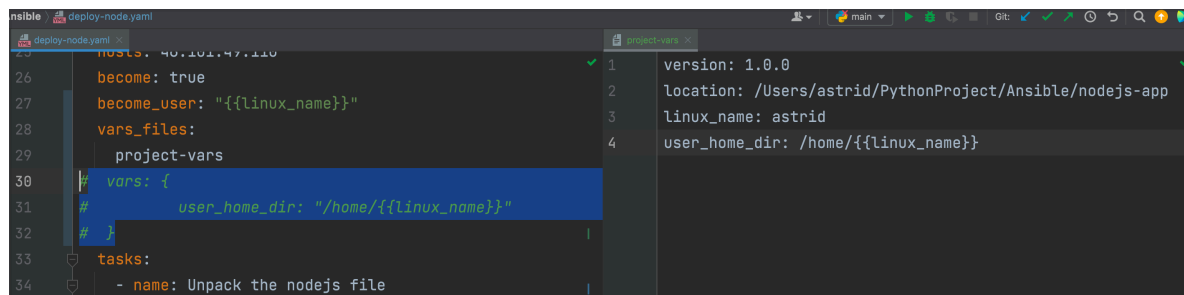
The screenshot shows an Ansible editor with two panels. The left panel displays a playbook named `deploy-node.yml` with the following content:

```
12
13 - name: Create new linux user for node app
14   hosts: 46.101.49.110
15   vars_files:
16     - project-vars
17   tasks:
18     - name: Create linux user
19       user:
20         name: "{{linux_name}}"
21         comment: Astrid admin
22         group: admin
23
24     - name: Deploy nodejs app
25       hosts: 46.101.49.110
26       become: true
27       become_user: "{{linux_name}}"
28       vars_files:
29         - project-vars
30       vars:
31         user_home_dir: "/home/{{linux_name}}"
32
33     tasks:
34       - name: Unpack the nodejs file
```

The right panel shows the `project-vars` file with the following content:

```
1 version: 1.0.0
2 location: /Users/astrid/PythonProject/Ansible/nodejs-app
3 linux_name: astrid
4
```

And we can also move the `user_home_dir` to the vars file



The screenshot shows the same Ansible editor with the updated files. The `deploy-node.yml` file now has:

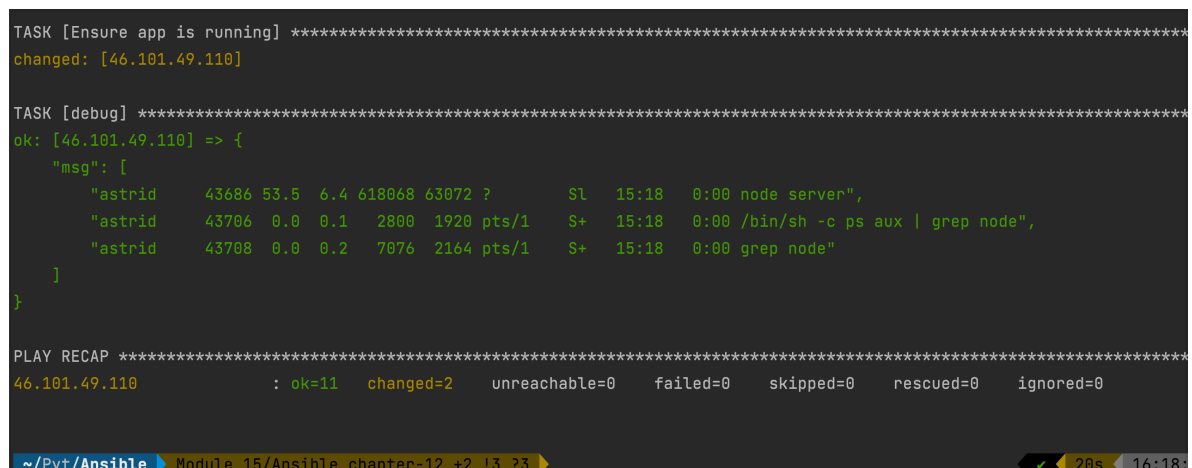
```
26 become: true
27 become_user: "{{linux_name}}"
28 vars_files:
29   - project-vars
30 vars:
31   user_home_dir: "/home/{{linux_name}}"
32
```

The `project-vars` file has been updated to include:

```
4 user_home_dir: /home/{{linux_name}}
```

And we execute the playbook again

-> `ansible-playbook -i hosts deploy-node.yml`



The screenshot shows the terminal output of the `ansible-playbook` command. It displays the execution of the `Unpack the nodejs file` task, which is successful. The output includes a debug message showing the command `ps aux | grep node` and its output. The final output shows the playbook recap:

```
TASK [Ensure app is running] *****
changed: [46.101.49.110]

TASK [debug] *****
ok: [46.101.49.110] => {
  "msg": [
    "astrid  43686 53.5  6.4 618068 63072 ?      Sl   15:18   0:00 node server",
    "astrid  43706 0.0  0.1  2800  1920 pts/1    S+   15:18   0:00 /bin/sh -c ps aux | grep node",
    "astrid  43708 0.0  0.2  7076  2164 pts/1    S+   15:18   0:00 grep node"
  ]
}

PLAY RECAP *****
46.101.49.110      : ok=11  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

Now every team member can have their own local version of the variables file:

